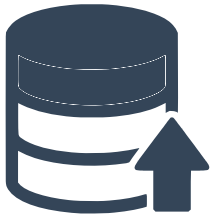


Structures de données linéaires



En informatique, une structure de données est une manière d'organiser les données dans le but de faciliter leurs traitements.

Pour prendre un exemple de la vie quotidienne, on peut présenter des numéros de téléphone par département, par nom, par profession (comme les Pages jaunes), par numéro téléphonique (comme les annuaires destinés au télémarketing), par rue et/ou une combinaison quelconque de ces classements. À chaque usage correspondra une structure d'annuaire appropriée.

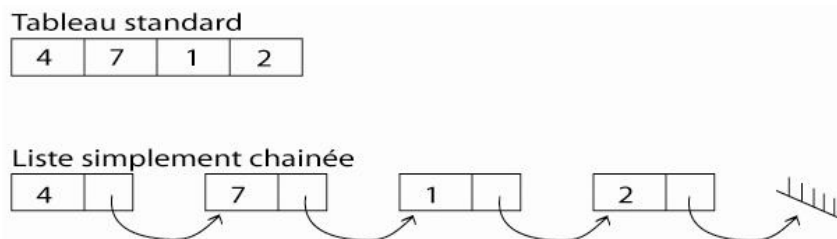
En organisant d'une certaine manière les données, on permet un traitement automatique de ces dernières plus efficace et plus rapide. On parle de **type abstrait de données (TAD)**.

Le fait d'utiliser une structure de données appropriée à un traitement informatique peut également faire baisser de manière significative la complexité d'une application informatique et ainsi contribuer à diminuer le taux d'erreurs.

La manipulation des types abstraits de données s'effectue à partir d'un ensemble de fonctions constituant l'interface de manipulation nommées primitives.

1. Les listes chaînées

L'implémentation d'une liste est efficace pour accéder à des éléments de cette même liste mais l'est beaucoup moins pour insérer des éléments. En effet insérer un élément oblige à décaler tous les éléments un par un. C'est pourquoi une autre implémentation est possible celle avec les listes chaînées.



Une liste simplement chaînée est une structure de données dans laquelle les objets sont arrangés linéairement. Toutefois, contrairement au tableau, pour lequel l'ordre linéaire est déterminé par les indices, l'ordre d'une liste chaînée est déterminé par un pointeur dans chaque objet. La modification d'un pointeur a un coût très inférieur à la réaffectation d'un élément d'où l'intérêt des liste chaînées.

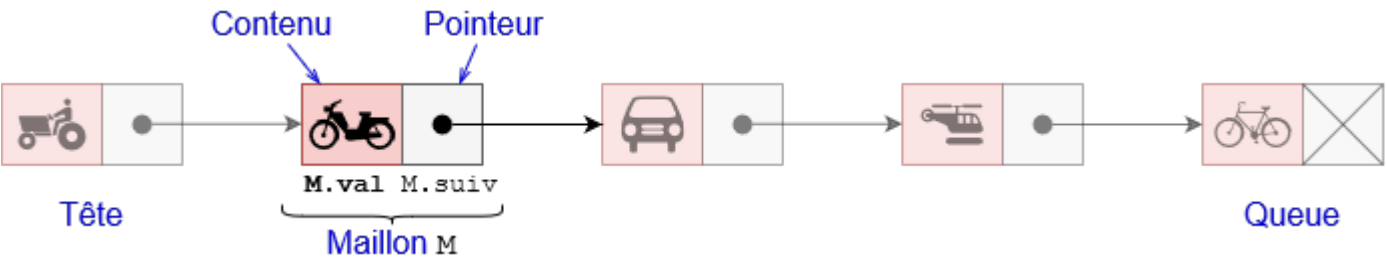
Chaque enregistrement d'une liste chaînée est souvent appelé élément, nœud ou maillon. La tête d'une liste est son premier nœud. La queue d'une liste peut se référer soit au reste de la liste après la tête, soit au dernier nœud de la liste.



Le champ de chaque nœud qui contient l'adresse du nœud suivant ou précédent est généralement appelé lien ou pointeur. Le contenu est placé dans un (ou plusieurs autres champs) appelés données, informations ou valeurs.

Chaque élément (ou maillon) M de la liste est composé :

- d'un contenu utile M.val (de n'importe quel type),
- d'un pointeur M.suiv pointant vers l'élément *suivant* de la séquence.



Le *dernier* élément de la liste possède un pointeur M.suiv vide : 

Une liste chaînée L est entièrement définie par son maillon de tête L.tete, c'est à dire l'adresse de son premier maillon.



Opérations de base

Il n'existe pas de normalisation pour les primitives de manipulation de liste chaînées; L'interface se compose généralement des éléments suivants :

Nom	Opération
est_vide(L)	Renvoie <i>vrai</i> si la liste L est vide
taille(L)	Renvoie le nombre d'éléments de la liste L
get_dernier_maillon(L)	Renvoie le dernier élément de la liste
get_maillon_indice(L, i)	Renvoie le maillon d'indice i
ajouter_debut(L, d)	Ajoute un maillon d au début de la liste L
ajouter_fin(L, d)	Ajoute un maillon d à la fin de la liste L
insérer_apres(L, i, M)	Insert un maillon à l'indice i
supprimer_apres(L, M)	Supprime le maillon qui suit le maillon M



2.Exemple de classe Liste et Element

Pour implémenter une liste chaînée avec des objets, nous allons définir deux classes : la classe Element qui représente les éléments de la liste et la classe Liste qui représente la liste elle-même.

La classe Element a deux attributs : valeur, la valeur de l'élément, et suivant, l'élément suivant ou None si c'est le dernier de la liste. La classe Liste a pour attribut le premier élément de la liste ou None si la liste est vide.

```
class Element:
    def __init__(self, valeur, suivant=None):
        self.valeur = valeur
        self.suivant = suivant

class Liste:
    def __init__(self):
        self.debut = None
    def est_vide(self):
        if self.debut == None:
            return 'vrai'

    def inserer(self, valeur):
        self.debut = Element(valeur, self.debut)
    def tete(self):
        return self.debut.valeur
    def queue(self):
        reste = Liste()
        reste.debut = self.debut.suivant
        return reste
    def elements(self):
        res = []
        e = self.debut
        while e is not None:
            res.append(e.valeur)
            e = e.suivant
        return res
```

3.Les Piles (Stack)

Pour beaucoup d'applications, les seules opérations à effectuer sur les listes sont des insertions et des suppressions aux extrémités. Dans les piles les insertions et les suppressions se font à une seule extrémité, appelée sommet de pile.

Exemples d'utilisation

- ➔ La fonction annuler sur un logiciel
- ➔ Pages web visitées sur un navigateur
- ➔ Dans un microprocesseur une pile est réservée dans la mémoire afin d'y ranger les arguments des sous programmes et les adresses de retour.



Fonctionnement d'une pile

Une pile est une structure linéaire de données (données rangées en ligne) qui fonctionne selon le principe suivant : le dernier élément entré sera le premier sorti. Les piles sont de ce fait aussi appelées piles **LIFO**, pour **Last-In-First-Out**.

Une bonne image pour représenter une pile est une pile d'assiettes : c'est en haut de la pile qu'il faut prendre ou mettre une assiette.

L'implémentation en mémoire de cette structure est très simple, elle est réalisée en affectant une zone mémoire contiguë de taille égale à la taille de la pile souhaitée. Le haut de la pile (position du dernier élément empilé) étant constamment repéré par un pointeur nommé 'pointeur de pile'.

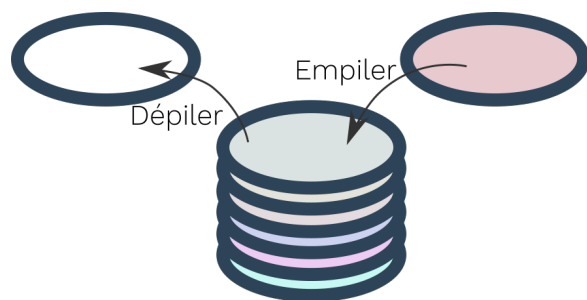


Figure 1: Fonctionnement d'une pile

Interface de manipulation

L'interface de manipulation d'une pile est composée habituellement des primitives suivantes :

Nom	Opérations	Implémentation Python à partir d'une liste
<code>creer_pile()</code>	Retourne une pile vide	<code>= []</code>
<code>est_vide(p)</code>	Teste si la pile p est vide. Renvoie True si vide.	<code>== []</code>
<code>empiler(p,e)</code>	Empile l'élément e sur la pile p	<code>.append(e)</code>
<code>depiler(p)</code>	Dépiler de p un élément	<code>.pop()</code>

Remarque : La méthode `pop()` a deux effets :

- x elle retire le dernier élément de la liste
- x elle renvoie la valeur de l'élément retiré

4. Les Files (Queue)

Une file dite aussi file d'attente est une structure de données basée sur le principe du premier entré, premier sorti désigné en anglais par l'acronyme FIFO (First In, First Out) . Pour une file, on parle de tête (1^{er} élément) et de queue (le reste) qui sont respectivement appelés head et tail en anglais.

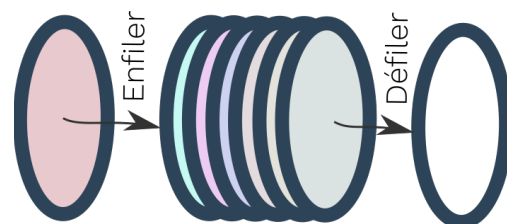


Figure 2: fonctionnement d'une file

Exemples d'utilisation

- x File d'attente d'un serveur d'impression qui traite les documents à imprimer au fur et à mesure où ils arrivent (spooler)
- x Mémoire tampon d'une carte réseau (buffer) qui mémorise les mots binaires transmis lorsqu'ils arrivent et les transmet au microprocesseur à la demande sous forme d'octets.



Opérations de base

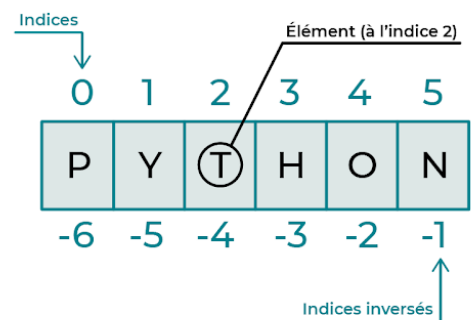
L'interface de manipulation d'une file est composée habituellement des primitives suivantes :

Nom	Opérations	Implémentation Python à partir d'une liste
<code>creer_file()</code>	Retourne une file vide	<code>= []</code>
<code>est_vide(p)</code>	Teste si la file f est vide. Renvoie True si vide.	<code>== []</code>
<code>enfiler(p,e)</code>	Enfile l'élément e sur la file f	<code>.append(e)</code>
<code>defiler(p)</code>	Défiler de f un élément	<code>.pop(0)</code>

5. Les listes

Une liste est une structure de données permettant de regrouper des données et dont l'accès est séquentiel. Elle correspond à une suite finie d'éléments repérés par leur rang (index). Les éléments sont ordonnés et leur place a une grande importance.

Une liste est évolutive, on peut ajouter ou supprimer un élément à n'importe quel endroit. Pour une liste aussi on peut parler de tête (1^{er} élément) et de queue (le reste).



Différences entre listes et tableaux

Un tableau est un ensemble de taille fixe contenant des valeurs de même type. Le fait que la taille ne varie pas durant son utilisation (interdiction de rajouter des éléments) facilite son implémentation en mémoire en allouant un espace mémoire contiguë.

Contrairement au tableau, la taille d'une liste peut varier au cours du temps. En effet, une liste n'est pas allouée en une seule fois, mais chaque élément est alloué indépendamment, sous la forme d'un maillon.

Temps d'accès à un élément dans un tableau ou une liste

Le temps d'accès à un élément d'un tableau par son index est constant, quel que soit l'élément désiré ($O(1)$). Cela s'explique par le fait que les éléments d'un tableau sont contigus dans l'espace mémoire. Ainsi, il est possible de calculer l'adresse mémoire de l'élément auquel on veut accéder, à partir de l'adresse de base du tableau et de l'index de l'élément. L'accès est immédiat, comme il le serait pour une variable simple.

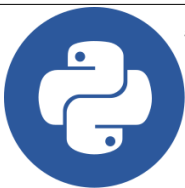
Dans une liste, l'accès à un élément nécessite de balayer tous les éléments à partir du premier jusqu'à l'indice voulu. Comparé à un tableau, ceci a pour effet de détériorer le temps d'accès ($O(n)$)



Opérations de base

L'interface de manipulation d'une liste est composée habituellement des primitives suivantes :

Nom	Opérations
<code>creer_liste()</code>	Retourne une liste vide
<code>est_vide(p)</code>	Teste si la list l est vide. Renvoie True si vide.
<code>insérer(l,e,i)</code>	Insère un élément e dans la liste l à l'index i
<code>supprimer(l,i)</code>	Supprime l'élément situé à l'index i de la liste l
<code>lire(l,i)</code>	Retourne l'élément situé à l'index i de la liste l
<code>modifier(l,e,i)</code>	Modifie l'élément situé à l'index i de la liste l avec la valeur e
<code>longueur(l)</code>	Renvoie le nombre d'éléments présents dans la liste l



Remarque : Les listes sous Python sont en fait implémentées dans des tableaux dont la taille est modifiée au fur et à mesure d'insertion de données. Ces tableaux se nomment des tableaux dynamiques. Ils offrent un meilleur temps d'accès lors d'une lecture/ écriture (temps constant) par contre ils nécessitent une recopie entière des données lors d'une nouvelle allocation d'espace du tableau.

6. Les dictionnaires

Un dictionnaire est une structure de données qui permet d'associer une valeur à une clé. Cette clé peut être un mot, un entier, un flottant ou même un tuple mais pas des listes. L'ensemble clé-valeur est appelée entrée ou enregistrement.

`d = {'a': 10, 'b': 20, 'c': 30}`

↑ ↑ ↑

`d['a']` `d['b']` `d['c']`

On rappelle que les dictionnaires ne comportent pas d'ordre contrairement aux listes. L'accès à un élément s'effectue donc à partir de sa clé et non pas grâce à un index.

Opérations de base

L'interface de manipulation d'une liste est composée habituellement des primitives suivantes :

Nom	Opérations
<code>creer_dico()</code>	Retourne une liste vide
<code>est_vide(d)</code>	Teste si la liste d est vide. Renvoie True si vide.
<code>insérer(d,cle, val)</code>	Insère l'ensemble cle, val dans le dico d
<code>supprimer(d,cle)</code>	Supprime l'enregistrement cle du dico d
<code>lire(d,cle)</code>	Retourne la valeur associée à la clé cle
<code>longueur(d)</code>	Renvoie le nombre d'enregistrement du dico d

